

GRUPO 5

# Quinto

# SPRINT

Havit

# Objetivos

1. Mejorar la calidad del código existente mediante la eliminación de componentes obsoletos y mejorar la estructura actual.
2. Incorporar pruebas unitarias automatizadas para validar el correcto funcionamiento de los módulos críticos del sistema
3. Incorporar funcionalidades sociales iniciales mediante el sistema de invitaciones a challenges y establecer la base del módulo de perfil para facilitar la implementación de seguidores, publicaciones y demás funcionalidades sociales en futuros sprints.
4. Calcular el costo del proyecto

# Backlog

## Bloque 1: Arquitectura y Limpieza

- Frontend: Reorganización de estructura y optimización de componentes reutilizables.
- Backend: Refactorización, limpieza de dependencias obsoletas y optimización.

## Bloque 2: Estrategia de Pruebas Unitarias

- Backend (Jest): Cobertura para lógica crítica y módulos secundarios.
- Frontend (Jest + RNRTL): Pruebas de componentes reutilizables e interfaz.
- Documentación: Definición de escenarios funcionales y evidencias de ejecución.

## Bloque 3: Módulo Social y Perfil

- Lógica y Endpoints: Gestión de invitaciones, estados y perfiles de usuario.
- Integración y UI: Diseño UX/UI, componentes de apoyo (popups/formularios) e integración completa.
- Infraestructura: Validación y carga de fotografías mediante Cloudflare R2.

# Limpieza de código

---

## Backend

- Accesos: login obligatorio por defecto en toda la API
- Propiedad: cada usuario solo toca sus propios datos (validado por token, no por lo que envía)
- Datos: dejó de exponerse el hash de contraseña; correo nunca visible a otros
- Orden: se eliminó el sistema de migraciones legacy; errores estandarizados

## Frontend

- Sesión protegida: token movido a almacenamiento cifrado del dispositivo
- Se eliminó un log con la contraseña en texto plano
- Orden: ~35 textos hardcodeados movidos a traducciones; se borraron mocks/datos falsos
- Simplificación: pantalla de 700+ líneas partida en componentes chicos

# Pruebas (Backend)

## Jest Framework

- Oficial para NestJS
- Soporte TypeScript
- Sin necesidad de instalar dependencias
- Mocks

## Pruebas realizadas

- AuthService
- ChallengeService
- WorkoutLogService
- MetricsService

```
PASS src/common/filters/http-exception.filter.spec.ts (5.045 s)
PASS src/auth/utils/assert-ownership.spec.ts
PASS src/auth/guards/jwt-auth.guard.spec.ts (5.314 s)
PASS src/challenge-invites/challenge-invites.service.spec.ts (5.896 s)
PASS src/auth/auth.service.spec.ts (5.951 s)
PASS src/metrics/metrics.service.spec.ts (5.992 s)
PASS src/exercises/exercises.service.spec.ts (6.031 s)
PASS src/challenges/challenges.service.spec.ts (6.213 s)
PASS src/workout-log/workout-log.service.spec.ts (6.291 s)
PASS src/users/users.service.spec.ts (6.377 s)
PASS src/workout-log/workout-log.controller.spec.ts (6.517 s)
```

```
Test Suites: 11 passed, 11 total
Tests:       94 passed, 94 total
Snapshots:   0 total
Time:        7.002 s
Ran all test suites.
```

# Pruebas (Frontend)

## Jest + React Native Testing Library

- Oficial para React Native
- Integración con Expo y TypeScript
- Pruebas de componentes e interfaz
- Mocks de servicios y funciones.

## Pruebas realizadas

- ActiveChallengeSection
- Home Screen
- hoursUntilMidnight()
- getHomeChallengesSorted()

```
PS C:\Users\Esteban\OneDrive\UVG\Semestre 5\old-havit\frontend> npx jest --runInBand
PASS components/home/__tests__/ActiveChallengeSection.test.tsx (6.786 s)
PASS app/(tabs)/__tests__/index.test.tsx
PASS services/adapters/__tests__/homeAdapter.test.ts
PASS utils/__tests__/time.test.ts
PASS i18n/__tests__/parity.test.ts

Test Suites: 5 passed, 5 total
Tests: 16 passed, 16 total
Snapshots: 0 total
Time: 12.131 s, estimated 21 s
Ran all test suites.
PS C:\Users\Esteban\OneDrive\UVG\Semestre 5\old-havit\frontend>
```

# Invitaciones y bases perfil

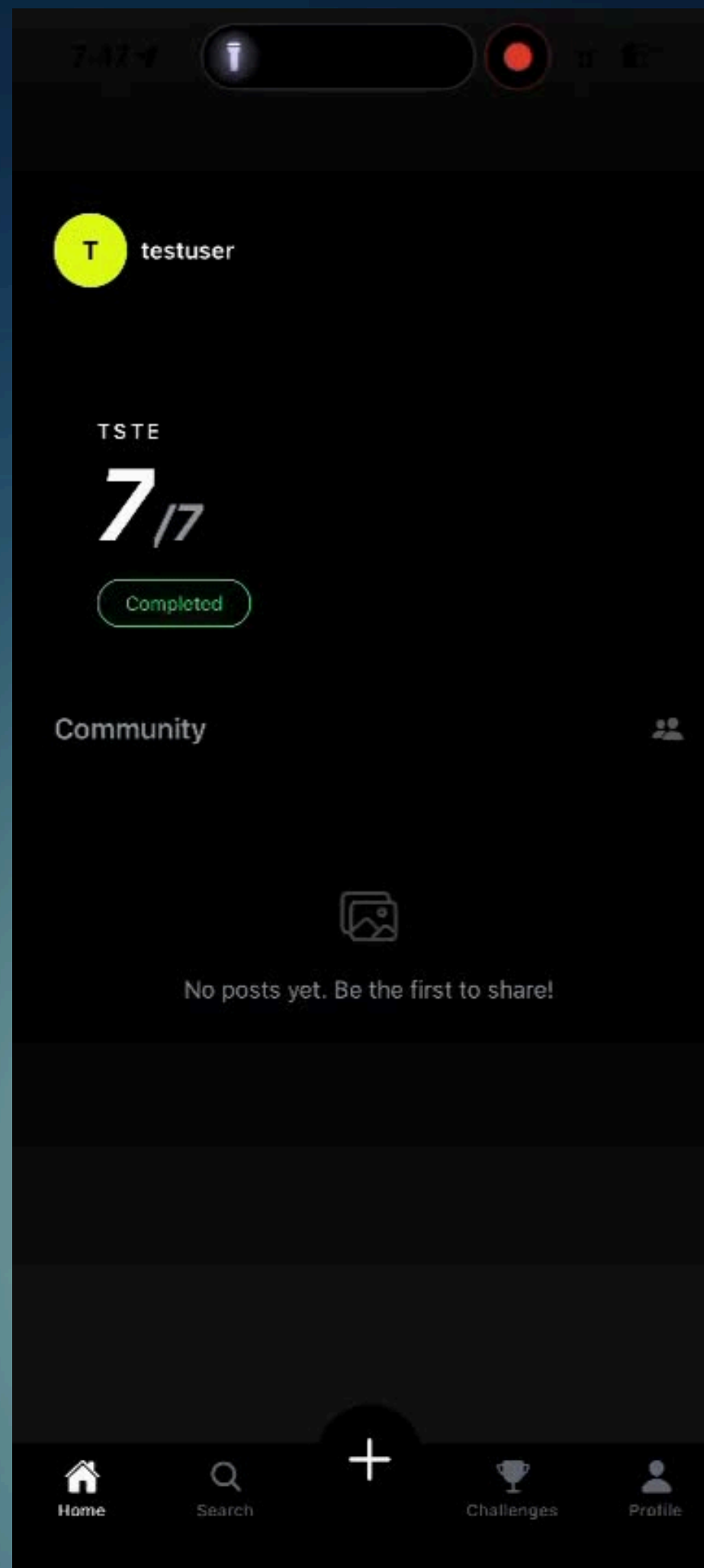
---

## Frontend

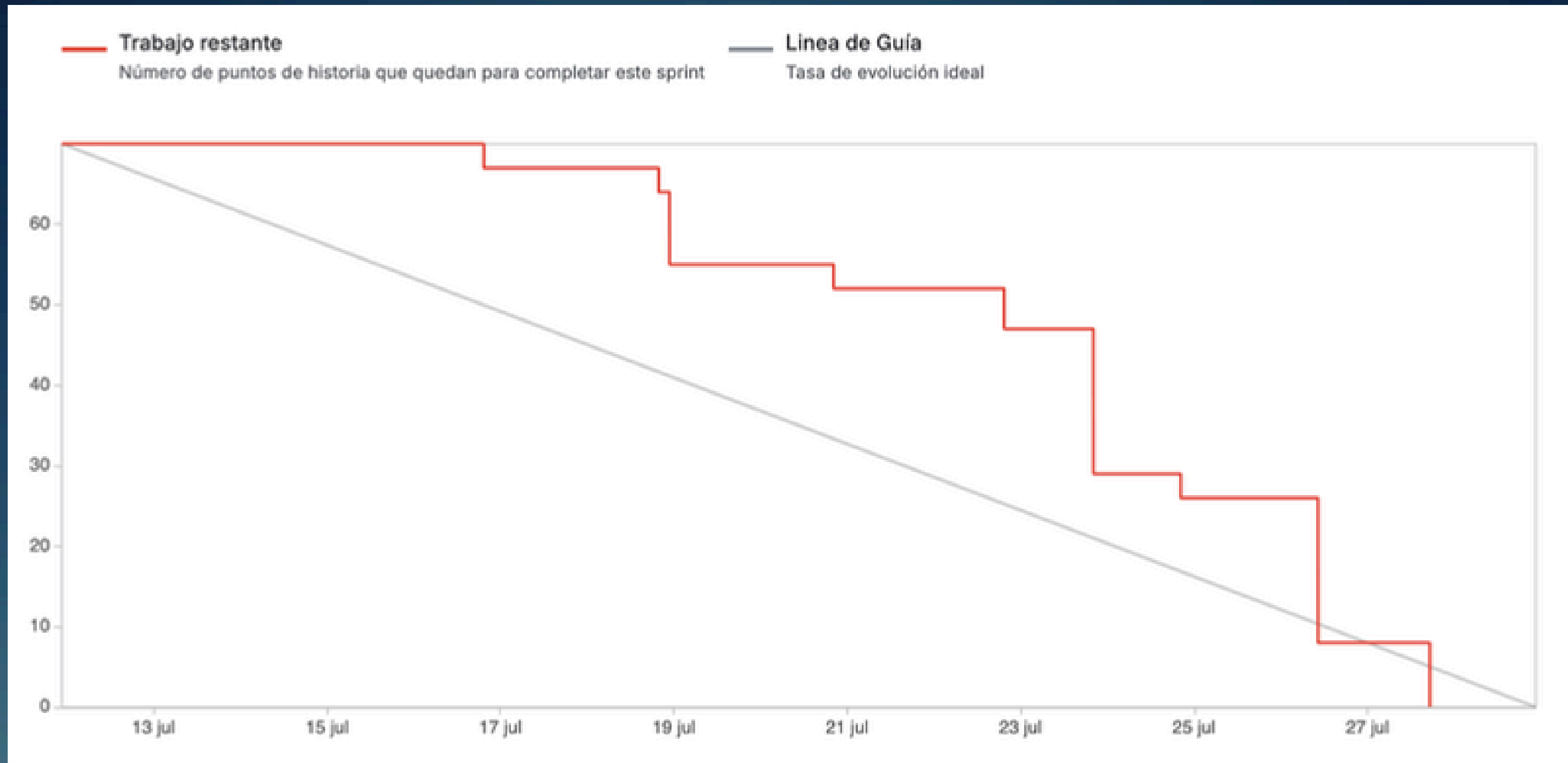
- Desarrollo de la funcionalidad completa de invitaciones a desafíos.
- Nuevas pantallas para invitaciones y edición de perfil.
- Creación de componentes reutilizables para formularios e invitaciones.
- Implementación de servicios y hooks para comunicación con el backend.
- Desarrollo de pruebas unitarias para componentes y servicios.

## Backend

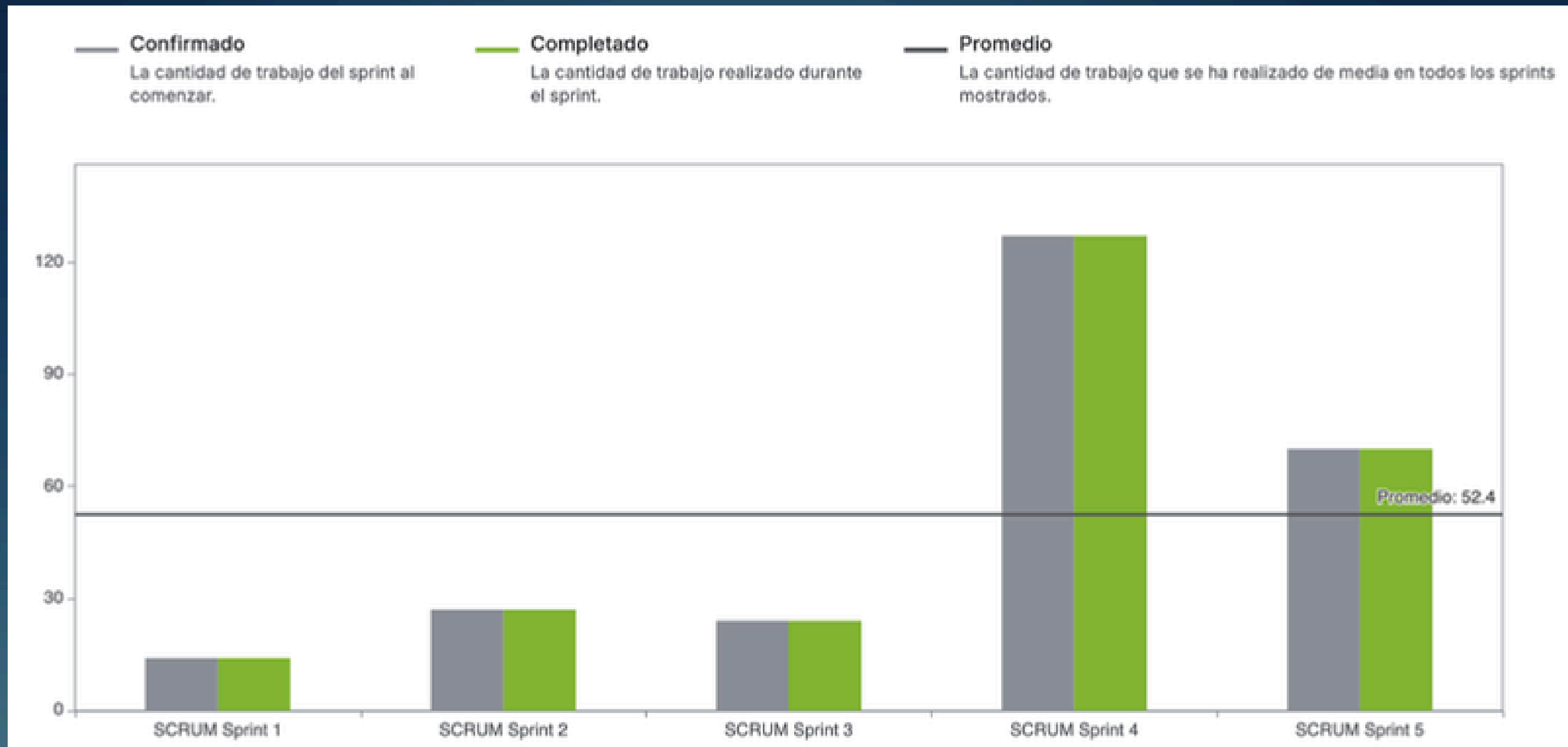
- Implementación del módulo de invitaciones a desafíos (Challenge Invites).
- Nuevos endpoints para crear, aceptar, rechazar y cancelar invitaciones.
- Desarrollo y conexión de la gestión de perfiles de usuario (edición de perfil, foto, privacidad y búsqueda de usuarios).
- Optimización de consultas mediante nuevos índices en base de datos.
- Implementación y actualización de pruebas unitarias para los nuevos módulos.



# Gráfico Burndown



# Velocidad de los Sprints



# Estimación de tiempo

---

## Técnica: Puntos de Casos de Uso (PCU)

- PCU: 90
- Factor técnico: 1.095
- Factor de ambiente: 0.89
- PCU ajustados: 87.71
- Esfuerzo total: 4,385.48 horas-hombre
- Equipo: 6 integrantes
- Duración del proyecto: 12 meses
- Dedicación promedio: 60.9 h/mes por integrante
- Aproximadamente 14 h semanales por integrante

## Resultado

Havit puede desarrollarse en 1 año con un equipo de seis personas trabajando a tiempo parcial.

# Estimación de costo

- Salario utilizado: Q4,252.28 mensuales
- Esfuerzo: 27.41 hombres-mes
- Costos indirectos:  $K = 1.5$
- Costo laboral: Q174,827.71

## Infraestructura por 1 año:

- Servidor Azure: US\$180
- Base de datos: US\$360
- Cloudflare R2: US\$12
- Total infraestructura:  
US\$552  $\approx$  Q4,207

## Costo total estimado: Q179,035

El salario usado corresponde al mínimo no agrícola de la Circunscripción Económica 1 más la bonificación de Q250. El cambio utilizado fue aproximadamente Q7.62 por dólar.

Los costos de infraestructura son estimados y deberán ajustarse al confirmar el proveedor y plan definitivo de la base de datos. El total no incluye margen de ganancia, soporte posterior ni crecimiento de infraestructura.

# Retrospectiva del Sprint

## QUE SALIÓ BIEN

- Se completaron todos los objetivos definidos.
- El trabajo en bloques permitió distribuir mejor las tareas.
- La retroalimentación de los usuarios fue positiva respecto a las nuevas funcionalidades.
- La limpieza y refactorización ayudó al orden del proyecto

## QUÉ SALIÓ MAL

- La limpieza tomó más de lo que planeamos
- Los comentarios de los usuarios evidenciaron oportunidades de mejora en la interfaz.

## ACCIONES

- Hacer revisiones al final de cada sprint para asegurar que se mantenga el orden
- Continuar refinando la interfaz de usuario tomando en cuenta la retroalimentación obtenida durante las pruebas con usuarios

# Conclusiones

---

## Conclusion 1

Se fortaleció la calidad del proyecto mediante la reorganización del código y la incorporación de pruebas unitarias, dejando una base más sólida para el desarrollo de futuras funcionalidades.

## Conclusion 2

Las nuevas funcionalidades de invitaciones y gestión del perfil ampliaron las capacidades sociales, mejorando la experiencia y la interacción de los usuarios dentro de la aplicación.

## Conclusion 3

Logramos completar los objetivos del sprint y mejoramos la planificación y coordinación durante el desarrollo.

## Conclusion 4

El análisis de costos permitió estimar que el desarrollo y operación de Havit durante su primer año requiere una inversión aproximada de Q179,033.



**GRACIAS**